# CS 432: Databases

## FixIIT – Maintenance Portal

Assignment 4: Sharding & Distributed Architecture


**Submitted by: Team Codebase**

Abhitej Singh Bhullar (23110009)
Shiv Patel (23110302)
Shivansh Shukla (23110303)
Soham Shrivastava (23110315)
Arin Mehta (23110038)


**GitHub Repository:** FixIIT Repository

**Video Demonstration:** A4 Demo Video

# Contents


**1 Introduction** **2**

**2 SubTask 1: Shard Key Selection & Justification** **2**
- 2.1 Chosen Shard Key: `member_id` . . . 2
- 2.2 Rejected Alternatives . . . 3
- 2.3 Partitioning Strategy: Deterministic Hash . . . 3

**3 SubTask 2: Implement Data Partitioning** **3**
- 3.1 Shard Isolation Environment . . . 3
- 3.2 Coordinator Schema . . . 3
- 3.3 Shard Table Schema . . . 4
- 3.4 Data Migration . . . 4
- 3.5 Verification: Post-Migration Counts . . . 5

**4 SubTask 3: Implement Query Routing** **5**
- 4.1 Insert Operations . . . 5
- 4.2 Point Lookups: Hash Routing . . . 6
- 4.3 Lookup by Ticket ID: Locator Routing . . . 6
- 4.4 Range Queries: Scatter-Gather Pattern . . . 6
- 4.5 Consistency Repair . . . 7

**5 SubTask 4: Scalability & Trade-offs Analysis** **8**
- 5.1 Horizontal vs. Vertical Scaling . . . 8
- 5.2 Consistency Analysis . . . 8
- 5.3 Availability Analysis . . . 8
- 5.4 Partition Tolerance . . . 9
- 5.5 CAP Theorem Classification . . . 9

**6 Performance Evaluation** **9**

**7 Verification & Audit** **10**
- 7.1 Migration Integrity . . . 10
- 7.2 Audit Trigger . . . 10

**8 Observations & Limitations** **10**

**9 Conclusion** **11**

# 1 Introduction

As database systems scale to support thousands of concurrent users, monolithic architectures suffer from I/O saturation and lock contention. A single database server becomes a bottleneck regardless of how well queries are optimised.

To address this, we transitioned the FixIIT campus maintenance portal from a single-node MySQL database into a **3-shard distributed system**. The implementation distributes the `tickets` table across three MySQL instances provided by the course infrastructure, coordinated by a central metadata node.

This report covers:

- Shard key selection and justification
- Shard table creation and data migration
- Application-level query routing logic
- Scalability and trade-off analysis
- Verification results and performance benchmarks

**Implementation Note:** Assignment 4 was implemented by directly modifying the existing `Assignment_2/Module_B` codebase (API, SQL, scripts, and related files). We did not create a separate `Assignment_4` source folder; instead, all sharding and distributed-system changes were integrated into the maintained Module B code. The Assignment 4 README documentation is included in `Assignment_2/Module_B/README.md`.

# 2 SubTask 1: Shard Key Selection & Justification

## 2.1 Chosen Shard Key: `member_id`

After evaluating candidate keys (`ticket_id`, `category_id`, `created_at`), we selected `member_id` for the following reasons:

1. **High Cardinality.** The FixIIT system serves students, faculty, and administrators across the entire institute. With hundreds of registered users, the `member_id` space is large enough to ensure statistically uniform distribution across any number of shards, avoiding data skew.

2. **Query Alignment.** Approximately 85% of application queries are user-scoped (e.g., *"My Tickets"*). These queries carry `member_id` in their `WHERE` clause, allowing the router to resolve the correct shard in $O(1)$ time without touching any other node.

3. **Stability / Immutability.** A `member_id` is an auto-generated primary key assigned at registration and never modified. A mutable shard key (e.g., `priority` or `status_id`) would require re-routing records across shards every time the value changes, which is prohibitively expensive.

### 2.2 Rejected Alternatives

| Candidate Key | Problem | Why Rejected |
|---|---|---|
| `ticket_id` | No query locality | Full-scatter for user views |
| `category_id` | Low cardinality (5 values) | Severe skew across shards |
| `created_at` | Mutable, range-dependent | Uneven growth over time |

Table 1: Rejected Shard Key Candidates

### 2.3 Partitioning Strategy: Deterministic Hash

We use modulo hashing for constant-time shard resolution:

$$\text{Shard Index} = (\texttt{member_id} - 1) \bmod 3 \tag{1}$$

The $-1$ offset ensures `member_id = 1` maps to Shard 0 rather than Shard 1, providing a more intuitive mapping. This strategy guarantees:

- No hotspot formation under uniform user activity
- Deterministic routing without any metadata lookup for user queries
- Constant-time shard resolution regardless of dataset size

**Expected distribution:** With 1000 tickets across a large, contiguous `member_id` range, each shard receives $\approx 333$ records ($\pm 5\%$).

## 3 SubTask 2: Implement Data Partitioning

### 3.1 Shard Isolation Environment

We used the instructor-provided MySQL instances hosted at `10.0.116.184`. Each shard runs as an independent MySQL server on a distinct port, providing physical process-level isolation:

| Role | Port | Database Name | Purpose |
|---|---|---|---|
| Coordinator | 3306 | `fixiit_db` | Metadata, ID allocation |
| Shard 0 | 3307 | `fixiit_ticket_shard_0` | `member_id % 3 = 0` |
| Shard 1 | 3308 | `fixiit_ticket_shard_1` | `member_id % 3 = 1` |
| Shard 2 | 3309 | `fixiit_ticket_shard_2` | `member_id % 3 = 2` |

Table 2: Distributed Node Topology

### 3.2 Coordinator Schema

The coordinator continues to host unsharded reference/auth tables from earlier assignments. For sharding, we introduced global ID allocation and locator metadata, along with migration-state and audit support.

Listing 1: Global ID Allocator (Coordinator)

```sql
CREATE TABLE ticket_id_allocator (
    ticket_id   INT AUTO_INCREMENT PRIMARY KEY,
    allocated_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Listing 2: Ticket Locator / Metadata Directory (Coordinator)

```sql
CREATE TABLE ticket_locator (
    ticket_id  INT      NOT NULL,
    member_id  INT      NOT NULL,
    shard_idx  TINYINT  NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (ticket_id),
    INDEX idx_locator_member (member_id),
    CONSTRAINT chk_shard_idx CHECK (shard_idx BETWEEN 0 AND 2)
) ENGINE=InnoDB;
```

### 3.3 Shard Table Schema

Each shard runs an identical `tickets` table. Indexes are tuned for the two dominant query patterns: user-scoped lookups and admin status filters.

Listing 3: tickets DDL (replicated on all 3 shards)

```sql
CREATE TABLE tickets (
    ticket_id   INT          NOT NULL,
    title       VARCHAR(120) NOT NULL,
    description TEXT,
    member_id   INT          NOT NULL,
    location_id INT,
    category_id INT,
    priority    VARCHAR(20),
    status_id   INT,
    created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  DATETIME     ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (ticket_id),
    INDEX idx_tickets_member_created (member_id, created_at),
    INDEX idx_tickets_status         (status_id)
) ENGINE=InnoDB;
```

### 3.4 Data Migration

Migration was performed by the script `migrate_tickets_to_shards.py`. The equivalent SQL for each shard is:

Listing 4: SQL Migration per Shard (example for Shard 0)

```sql
-- Shard 0: member_id where (member_id - 1) % 3 = 0
INSERT INTO fixiit_ticket_shard_0.tickets
SELECT ticket_id, title, description, member_id,
       location_id, category_id, priority,
       status_id, created_at, updated_at
FROM   fixiit_db.tickets
WHERE  (member_id - 1) % 3 = 0;

-- Populate locator for Shard 0 records
INSERT INTO fixiit_db.ticket_locator (ticket_id, member_id, shard_idx)
```

```sql
SELECT ticket_id, member_id, 0
FROM   fixiit_db.tickets
WHERE  (member_id - 1) % 3 = 0;
```

The same pattern repeats with `= 1` and `= 2` for Shards 1 and 2.

### 3.5 Verification: Post-Migration Counts

After migration, the following `SELECT COUNT(*)` results were recorded:

| Shard | Port | Ticket Count | Distribution |
|---|---|---|---|
| Shard 0 | 3307 | 342 | 34.2% |
| Shard 1 | 3308 | 338 | 33.8% |
| Shard 2 | 3309 | 320 | 32.0% |
| **Total** | | **1000** | **100.0%** |

Table 3: Verified Ticket Distribution – No Duplicates, No Data Loss

A cross-verification query confirmed that no `ticket_id` appears in more than one shard and the total matches the pre-migration source count of 1000.

## 4 SubTask 3: Implement Query Routing

Routing is implemented entirely at the **application layer** (Python/Flask API). No middleware proxy (e.g., ProxySQL) is used. This gives full control over routing logic but places the coordination burden on the application.

### 4.1 Insert Operations

Every new ticket insert follows a two-phase protocol:

1. Allocate a globally unique `ticket_id` from the coordinator's `ticket_id_allocator` table.
2. Compute target shard via (`member_id` − 1) mod 3, write the record to that shard, and update the locator directory.

Listing 5: Insert Routing Logic

```python
def insert_ticket(member_id, title, description, ...):
    # Step 1: allocate global ID from coordinator
    ticket_id = coordinator.execute(
        "INSERT INTO ticket_id_allocator () VALUES (); "
        "SELECT LAST_INSERT_ID();"
    )

    # Step 2: compute target shard
    shard_idx = (member_id - 1) % 3
    shard_conn = get_shard_connection(shard_idx)

    # Step 3: write to shard
    shard_conn.execute("""
```

```
        INSERT INTO tickets
        (ticket_id, title, description, member_id, ...)
        VALUES (%s, %s, %s, %s, ...)
    """, (ticket_id, title, description, member_id, ...))

    # Step 4: update locator
    coordinator.execute("""
        INSERT INTO ticket_locator (ticket_id, member_id, shard_idx)
        VALUES (%s, %s, %s)
    """, (ticket_id, member_id, shard_idx))
```

### 4.2 Point Lookups: Hash Routing

When `member_id` is available in the session (all user-facing routes), the router resolves the shard in $O(1)$ without touching the coordinator:

Listing 6: Hash-Based Point Lookup

```
def get_user_tickets(member_id):
    shard_idx = (member_id - 1) % 3
    conn = get_shard_connection(shard_idx)
    return conn.execute(
        "SELECT * FROM tickets WHERE member_id = %s "
        "ORDER BY created_at DESC",
        (member_id,)
    ).fetchall()
```

### 4.3 Lookup by Ticket ID: Locator Routing

When only `ticket_id` is known (e.g., admin drill-down), the coordinator locator is queried first:

Listing 7: Locator-Based Routing

```
def get_ticket_by_id(ticket_id):
    row = coordinator.execute(
        "SELECT shard_idx FROM ticket_locator WHERE ticket_id = %s",
        (ticket_id,)
    ).fetchone()

    if row is None:
        # Locator stale: scan all shards to repair
        return repair_and_fetch(ticket_id)

    shard_idx = row['shard_idx']
    conn = get_shard_connection(shard_idx)
    return conn.execute(
        "SELECT * FROM tickets WHERE ticket_id = %s",
        (ticket_id,)
    ).fetchone()
```

### 4.4 Range Queries: Scatter-Gather Pattern

Admin queries (date ranges, status filters) cannot be resolved to a single shard. The router queries all shards and merges results in memory:

Listing 8: Scatter-Gather for Admin Range Queries

```python
def get_admin_tickets(created_from=None, created_to=None,
                      ticket_id_min=None, ticket_id_max=None):
    results = []
    clauses, params = [], []

    if created_from is not None:
        clauses.append("created_at >= %s")
        params.append(created_from)
    if created_to is not None:
        clauses.append("created_at <= %s")
        params.append(created_to)
    if ticket_id_min is not None:
        clauses.append("ticket_id >= %s")
        params.append(ticket_id_min)
    if ticket_id_max is not None:
        clauses.append("ticket_id <= %s")
        params.append(ticket_id_max)

    query = "SELECT * FROM tickets"
    if clauses:
        query += " WHERE " + " AND ".join(clauses)
    query += " ORDER BY created_at DESC, ticket_id DESC"

    # Query all shards and merge
    for shard_idx, config in enumerate(SHARD_CONFIGS):
        partial = fetch_all(query, tuple(params), db_config=config)
        results.extend(partial)

    # Global merge-sort by timestamp and ticket id descending
    results.sort(key=lambda r: (r['created_at'], r['ticket_id']),
        reverse=True)
    return results
```

**Trade-off:** Scatter-gather queries are inherently slower than single-shard lookups, as they require $N$ network round trips and an in-memory sort. This is acceptable for admin views, which are low-frequency and high-privilege operations.

### 4.5 Consistency Repair

If the locator table becomes stale (e.g., due to a partial write failure), the system falls back to a full shard scan and repairs the locator entry:

Listing 9: Locator Repair on Cache Miss

```python
def repair_and_fetch(ticket_id):
    for shard_idx, config in enumerate(SHARD_CONFIGS):
        row = fetch_one(
            "SELECT * FROM tickets WHERE ticket_id = %s",
            (ticket_id,), db_config=config
        )
        if row:
            # Repair locator
            coordinator.execute("""
                INSERT INTO ticket_locator (ticket_id, member_id,
                    shard_idx)
                VALUES (%s, %s, %s)
                ON DUPLICATE KEY UPDATE shard_idx = VALUES(shard_idx)
```

```
            """, (ticket_id, row['member_id'], shard_idx))
            return row
    return None
```

# 5 SubTask 4: Scalability & Trade-offs Analysis

## 5.1 Horizontal vs. Vertical Scaling

**Vertical scaling** (scaling up) means upgrading a single server with more RAM, faster disks, or more CPU cores. It is operationally simple but has a hard physical ceiling—at some point, hardware limits cannot be exceeded at any cost. It also creates a single point of failure.

**Horizontal scaling** (sharding) distributes data across multiple independent nodes. Each node handles a subset of the load, so throughput scales approximately linearly with the number of shards added. In our system, tripling the number of shards would ideally triple the write throughput for user-scoped operations. The cost of this is architectural complexity: cross-shard queries, distributed ID allocation, and metadata management all require custom application logic that a single-node system never needs.

## 5.2 Consistency Analysis

- **Within a shard**: Full ACID guarantees via InnoDB. A ticket insert is atomic at the shard level.
- **Across shards**: Eventual consistency only. The insert protocol (shard write then locator write) is not wrapped in a two-phase commit (2PC). A crash between the shard write and the locator write leaves the locator temporarily stale. The repair mechanism corrects this lazily on the next read.
- **No distributed transactions**: 2PC was not implemented due to its significant performance and complexity overhead for this scale. This is an acceptable trade-off given that FixIIT is not a financial system requiring strict cross-shard atomicity.

## 5.3 Availability Analysis

If one shard goes offline:

- Users whose `member_id` hashes to the failed shard cannot access their tickets. Their requests fail with an API error response.
- Users on the remaining two shards are **completely unaffected** — their queries never touch the failed node.
- Admin scatter-gather queries depend on all shard reads. If one shard read fails, the admin endpoint fails the request rather than returning partial data.
- The coordinator (Port 3306) remains a single point of failure. If it fails, all inserts and locator-based lookups are blocked, even if all shards are healthy.

This design provides **partial availability**: the fraction of users affected by a shard failure is $\approx 1/3$, compared to 100% unavailability in a monolithic system.

### 5.4 Partition Tolerance

When a shard becomes unreachable due to a network partition:

- The application detects shard unreachability when the shard DB operation fails and returns an error response from the API layer.
- Hash-routed user queries to the affected shard fail fast with a clear error.
- Locator-repair logic is best-effort for stale metadata; it does not provide write availability when the target shard itself is unreachable.
- Once the shard reconnects, normal routing resumes immediately because shard assignment is deterministic from `member_id`.

### 5.5 CAP Theorem Classification

Our system prioritises **Availability (A)** and **Partition Tolerance (P)**, accepting **Eventual Consistency** rather than strong global consistency. Under a network partition, the system continues serving requests for unaffected shards rather than refusing all writes to preserve global consistency.

## 6 Performance Evaluation

Benchmarks were recorded as the average of 100 repeated queries on a local development machine connected to the institute network. "Before" reflects the monolithic single-node schema from Assignment 2. "After" reflects the sharded system with revised indexes.

| Query Type | Before (ms) | After (ms) | Improvement | Routing Type |
|---|---|---|---|---|
| LOGIN_QUERY | 0.155 | 0.098 | 36.94% | Single shard (hash) |
| ADMIN_CHECK_QUERY | 0.069 | 0.059 | 15.17% | Coordinator lookup |
| USER_TICKETS_QUERY | 0.072 | 0.065 | 9.35% | Single shard (hash) |
| ADMIN_TICKETS_QUERY | 0.119 | 0.111 | 7.01% | Scatter-gather |

Table 4: Measured Query Performance Before and After Sharding

**Observations:**

- Login and user-ticket queries show the largest gains because they are now single-shard operations scanning a smaller dataset with a compound index.
- Admin ticket queries show the smallest gain because scatter-gather introduces additional network overhead that partially offsets the per-shard reduction in scan cost.
- Performance improvements are a combined effect of sharding and the revised compound index `(member_id, created_at)`, which replaces the previous single-column index.

# 7 Verification & Audit

## 7.1 Migration Integrity

Post-migration verification confirmed:

- Total records across all shards: 1000 (matches source exactly)
- No duplicate `ticket_id` values across shards (verified via full cross-shard `UNION` count)
- Shard placement correct: every record's `member_id` satisfies (`member_id`$-1$) mod 3 = `shard_idx`
- Locator table fully populated: 1000 rows, one per ticket

## 7.2 Audit Trigger

Audit logging is implemented using `AFTER INSERT/UPDATE/DELETE` triggers on `tickets` (each shard) and on coordinator tables such as `ticket_locator`/`member_portfolio`. Triggers write event metadata into `db_change_audit` and classify source as `API` vs `DIRECT_DB` using `@app_actor_member_id` and `@app_endpoint` context variables.

Representative trigger logic:

Listing 10: Write Source Audit Trigger

```sql
CREATE TRIGGER trg_tickets_ai_audit
AFTER INSERT ON tickets
FOR EACH ROW
BEGIN
    INSERT INTO db_change_audit (
        table_name, operation, pk_value, actor_member_id,
        endpoint, source, before_json, after_json, changed_at
    )
    VALUES (
        'tickets',
        'INSERT',
        CAST(NEW.ticket_id AS CHAR),
        @app_actor_member_id,
        @app_endpoint,
        CASE
            WHEN @app_actor_member_id IS NULL OR @app_endpoint IS NULL
                THEN 'DIRECT_DB'
            ELSE 'API'
        END,
        NULL,
        JSON_OBJECT('ticket_id', NEW.ticket_id, 'member_id', NEW.
            member_id),
        NOW()
    );
END;
```

# 8 Observations & Limitations

- **Coordinator bottleneck**: Every insert requires a round trip to Port 3306 for ID allocation. Under very high write concurrency, this becomes a throughput ceiling.

A future improvement would be to use UUID-based IDs generated client-side, eliminating the coordinator write path for inserts.

- **No distributed transactions**: The two-phase insert (shard write + locator write) is not atomic. A process crash between the two steps leaves an orphaned ticket not visible via locator lookup until the repair path runs.
- **Re-sharding complexity**: Adding a fourth shard would require remapping $\approx 25\%$ of existing records. There is no live migration path; the system would require downtime to redistribute data.
- **Cross-shard joins not supported**: Queries that join `tickets` with tables on other shards (e.g., `comments`) must either denormalise data into the shard or perform application-level joins, which are expensive.
- **Scatter-gather latency scales linearly**: As the number of shards grows, admin range queries take proportionally longer. This is the inherent cost of horizontal partitioning for non-shard-key queries.

## 9 Conclusion

The FixIIT distributed system successfully implements horizontal sharding across three MySQL instances using deterministic hash partitioning on `member_id`. The hybrid routing strategy (hash routing for user queries, locator routing for admin queries, scatter-gather for range queries) provides correct and efficient query execution across all API endpoints.

The implementation demonstrates measurable performance improvements for single-shard queries while clearly identifying the inherent trade-offs of distributed architecture: partial availability, eventual consistency, coordinator bottleneck, and re-sharding complexity. These trade-offs are well-understood and represent standard characteristics of sharded systems at this scale.